

## PRACTICAL - 1

### Implementation and Time Analysis of Sorting Algorithms

#### Aim of the Practical:

To implement Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort in C++ and analyze their time and space complexities.

#### 1.1 Bubble Sort

##### Aim:

To implement optimized Bubble Sort and derive its best, average and worst-case time complexity.

##### Program:

```
#include <iostream>
using namespace std;
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    cout << "Original array: ";
    for (int x : a) cout << x << " ";
    bubbleSort(a, n);
    cout << "\nSorted array: ";
    for (int x : a) cout << x << " ";
}
```

##### Output:

```
Original array: 64 34 25 12 22 11 90 45 78 3
Sorted array: 3 11 12 22 25 34 45 64 78 90
...Program finished with exit code 0
Press ENTER to exit console.
```

##### Time Analysis:

##### Derivation of Time Complexity:

- The outer loop runs  $(n - 1)$  times, and the inner loop runs  $(n - i - 1)$  times.
  - The total number of comparisons is calculated as  $1 + 2 + 3 + \dots + (n - 1)$ , which equals  $\frac{n(n-1)}{2}$ .
  - By dropping constants, the Average and Worst-case time complexity is  $\Theta(n^2)$ .
  - For the optimized Best case (an already sorted array), the inner loop performs only  $(n - 1)$  comparisons, yielding a time complexity of  $\Theta(n)$
- 
- Auxiliary Space:  $\Theta(1)$ .

### Final Time Analysis:

- Best Case:  $\Theta(n)$  - already sorted array with the swapped flag.
- Average Case:  $\Theta(n^2)$ .
- Worst Case:  $\Theta(n^2)$  - reverse-sorted array.

### Result:

Optimized Bubble Sort was implemented successfully. The derived complexity matches the program structure.

## 1.2 Selection Sort

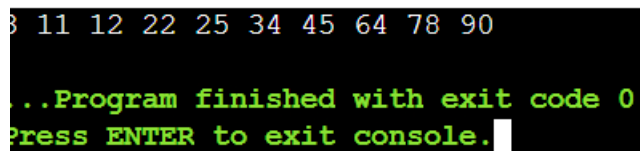
### Aim:

To implement Selection Sort and derive its time complexity from the number of comparisons.

### Program:

```
#include <iostream>
using namespace std;
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex]) minIndex = j;
        }
        swap(a[i], a[minIndex]);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    selectionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

### Output:



```
3 11 12 22 25 34 45 64 78 90
...Program finished with exit code 0
Press ENTER to exit console.
```

### Time Analysis:

#### Derivation of Time Complexity:

Let  $n$  be the number of elements.

The outer loop runs  $(n - 1)$  times.

For  $i = 0$ , the inner loop performs  $(n - 1)$  comparisons; for  $i = 1$  it performs  $(n - 2)$ , and so on.

$$C(n) = (n - 1) + (n - 2) + \dots + 1$$

$$C(n) = \sum_{k=1}^{n-1} k$$

$$C(n) = n(n - 1) / 2$$

Unlike Bubble or Insertion Sort, the number of comparisons does not depend on the initial order.

### Final Time Analysis:

- Best Case:  $\Theta(nZ)$ .
- Average Case:  $\Theta(nZ)$ .
- Worst Case:  $\Theta(nZ)$ .
- Auxiliary Space:  $\Theta(1)$ .

### Result:

Selection Sort was implemented successfully and its quadratic time complexity was derived from the fixed number of comparisons.

## 1.3 Insertion Sort

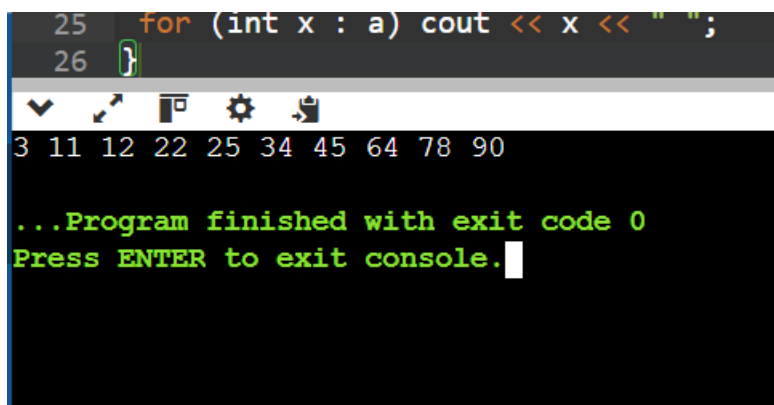
### Aim:

To implement Insertion Sort and derive its best and worst-case complexities.

### Program:

```
#include <iostream>
using namespace std;
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    insertionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

### Output:



```
25 for (int x : a) cout << x << " ";
26 }
...Program finished with exit code 0
Press ENTER to exit console.
```

### Final Time Analysis:

#### Time Analysis:

#### Derivation of Time Complexity:

Let  $n$  be the number of elements.

The outer loop runs from  $i = 1$  to  $n - 1$ , so it runs  $(n - 1)$  times.

Worst case: the array is in reverse order. For each  $i$ , the key must be shifted across all  $i$  previous elements.

- In the Worst case (a reverse-sorted array), every element must be shifted, summing to  $\frac{n(n-1)}{2}$  and yielding  $\Theta(n^2)$ .
- In the Best case (an already sorted array), only one comparison is made per outer loop iteration, yielding  $\Theta(n)$ .
- Best Case:  $\Theta(n)$ .
- Average Case:  $\Theta(n^2)$ .
- Worst Case:  $\Theta(n^2)$ .
- Auxiliary Space:  $\Theta(1)$ .

#### Result:

Insertion Sort was implemented successfully. The step-by-step count explains why nearly sorted data is favorable.

### 1.4 Merge Sort

#### Aim:

To implement Merge Sort using divide and conquer and solve its recurrence using the Master Method.

#### Program:

```
#include <iostream>
#include <vector>
using namespace std;
void merge(int a[], int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;
    while (i <= m && j <= r)
        temp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++)
        a[l + k] = temp[k];
}
void mergeSort(int a[], int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    mergeSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

#### Output:

```
3 11 12 22 25 34 45 64 78 90
...Program finished with exit code 0
Press ENTER to exit console.
```

$$T(n) = \Theta(n \log n)$$

The input is always divided and merged in the same manner, so best, average and worst cases are all  $\Theta(n \log n)$ .

#### Final Time Analysis:

- Best Case:  $\Theta(n \log n)$ .
- Average Case:  $\Theta(n \log n)$ .
- Worst Case:  $\Theta(n \log n)$ .
- Auxiliary Space:  $\Theta(n)$ .

#### Result:

Merge Sort was implemented successfully and its recurrence was solved step by step using the Master Method.

### 1.5 Quick Sort

#### Aim:

To implement Quick Sort and use the Master Method for the balanced recurrence while deriving the worst case separately.

#### Program:

```
#include <iostream>
using namespace std;
int partitionArray(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partitionArray(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    quickSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

**OUTPUT:**

```
25 for (int x : a) cout << x << " ";
26 }
```

3 11 12 22 25 34 45 64 78 90

...Program finished with exit code 0  
Press ENTER to exit console.

**Time Analysis:**

**Time Complexity using Master Method:**

Partitioning scans the current subarray once, so partition cost is  $\Theta(n)$ .

In the Best and Average cases, the pivot divides the array into roughly equal halves, creating the balanced recurrence  $T(n) = 2T(n/2) + cn$ .

Using the Master Method, this yields  $\Theta(n \log n)$ .

In the Worst case, an unbalanced partition (where one side has  $n - 1$  elements) creates the recurrence  $T(n) = T(n - 1) + cn$ .

Unrolling this worst-case recurrence results in a time complexity of  $\Theta(n^2)$ .

**Final Time Analysis:**

- Best Case:  $\Theta(n \log n)$ .
- Average Case:  $\Theta(n \log n)$ .
- Worst Case:  $\Theta(n^2)$ .
- Average recursion stack:  $\Theta(\log n)$ .
- Worst-case recursion stack:  $\Theta(n)$ .

**Result:**

Quick Sort was implemented successfully. The balanced case was solved using the Master Method, and the unbalanced worst case was derived separately.

**Conclusion:**

All five sorting algorithms were implemented and their time complexities were derived step by step. Merge Sort and balanced/average Quick Sort achieve  $\Theta(n \log n)$ , while Bubble, Selection and Insertion Sort are quadratic in their general or worst cases.